

## Industrial Internship Report on " URL Shortener"

Prepared by  
**Aditya Kumar**

### *Executive Summary*

The **URL Shortener** project is a web-based application developed using **Python** and the **Flask** framework to simplify the process of sharing long web addresses by converting them into short, unique, and user-friendly URLs. Long URLs are often difficult to remember, share, and manage, especially on social media platforms, emails, and messaging applications. This project addresses that challenge by providing a lightweight, efficient, and easy-to-use URL shortening solution.

The application accepts a valid long URL from the user, verifies its format using URL validation techniques, generates a unique six-character short code, and stores the mapping between the original URL and the shortened URL in an **SQLite** database. Whenever the shortened URL is accessed, the system retrieves the corresponding original URL from the database and automatically redirects the user to the intended webpage.

The project was implemented using **Python, Flask, SQLite, HTML, and CSS**, following a modular development approach. The application was thoroughly tested to ensure accurate URL validation, reliable database storage, successful URL generation, and seamless redirection. The testing results demonstrated that the system performs efficiently while maintaining simplicity and ease of use.

This project provided practical exposure to **Python programming, Flask web development, database management, routing, form handling, error handling, and software testing**. It also enhanced problem-solving, debugging, and software documentation skills.

## TABLE OF CONTENTS

|     |                                              |                                     |
|-----|----------------------------------------------|-------------------------------------|
| 1   | Preface .....                                | 3                                   |
| 2   | Introduction .....                           | 4                                   |
| 2.1 | About UniConverge Technologies Pvt Ltd ..... | 4                                   |
| 2.2 | About upskill Campus .....                   | 7                                   |
| 2.3 | Objective .....                              | 9                                   |
| 2.4 | Reference.....                               | 9                                   |
| 2.5 | Glossary.....                                | 9                                   |
| 3   | Problem Statement .....                      | 10                                  |
| 4   | Existing and Proposed solution.....          | 10                                  |
| 5   | Proposed Design/ Model .....                 | 12                                  |
| 6   | Performance Test.....                        | 12                                  |
| 6.1 | Test Plan/ Test Cases .....                  | 13                                  |
| 6.2 | Test Procedure .....                         | 14                                  |
| 6.3 | Performance Outcome .....                    | 14                                  |
| 7   | My learnings.....                            | <b>Error! Bookmark not defined.</b> |
| 8   | Future work scope .....                      | 16                                  |

## 1 Preface

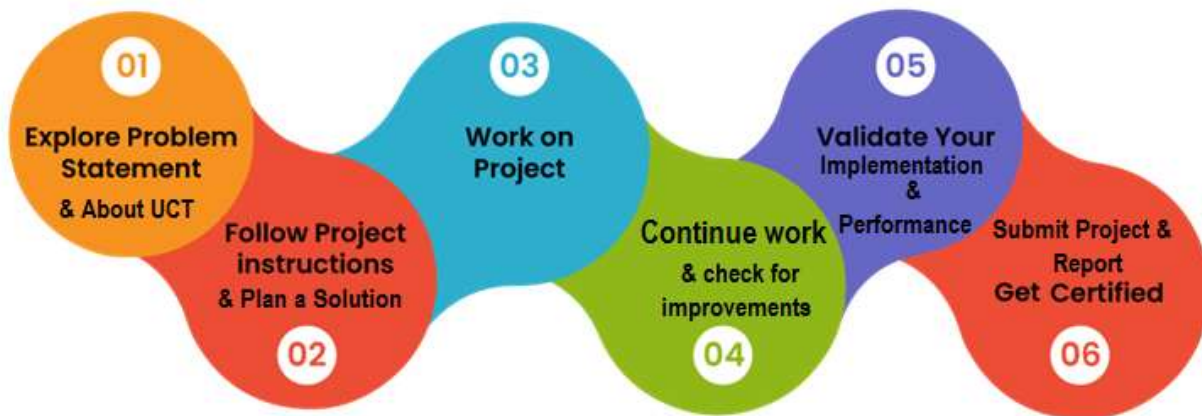
Summary of the whole 6 weeks' work.

About need of relevant Internship in career development.

Brief about Your project/problem statement.

Opportunity given by USC/UCT.

How Program was planned



Your Learnings and overall experience.

Thank to all (with names), who have helped you directly or indirectly.

Your message to your juniors and peers.

## 2 Introduction

### 2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies** e.g. **Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



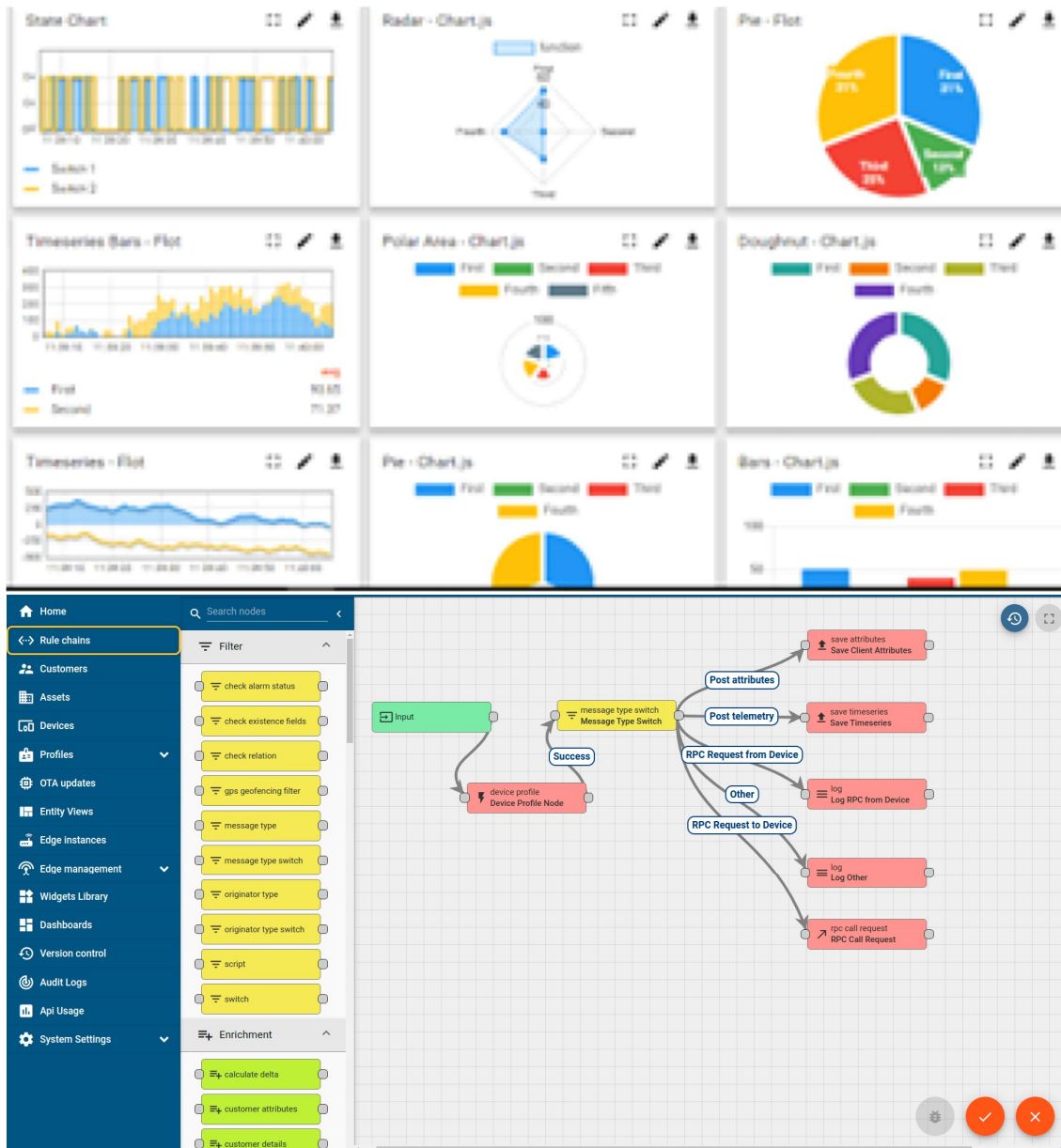
#### i. UCT IoT Platform ( )

**UCT Insight** is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



# FACTORY WATCH

## ii. Smart Factory Platform ( )

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleashed the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.





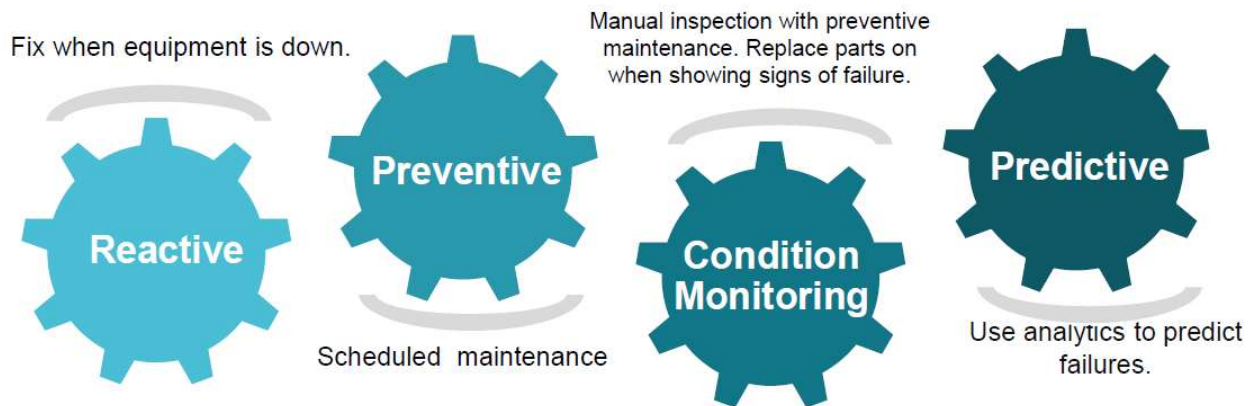


### iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN technology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

### iv. Predictive Maintenance

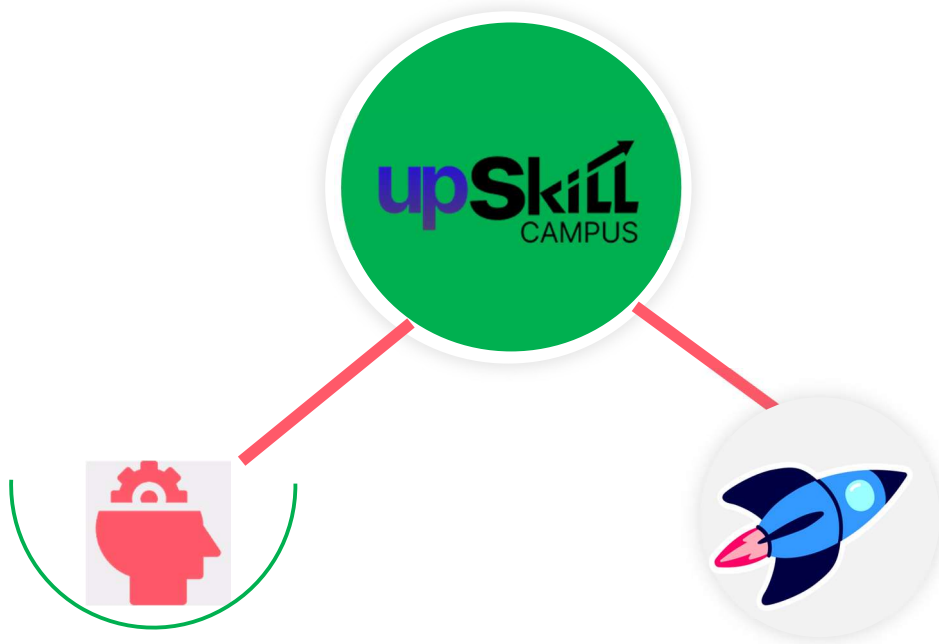
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



## 2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

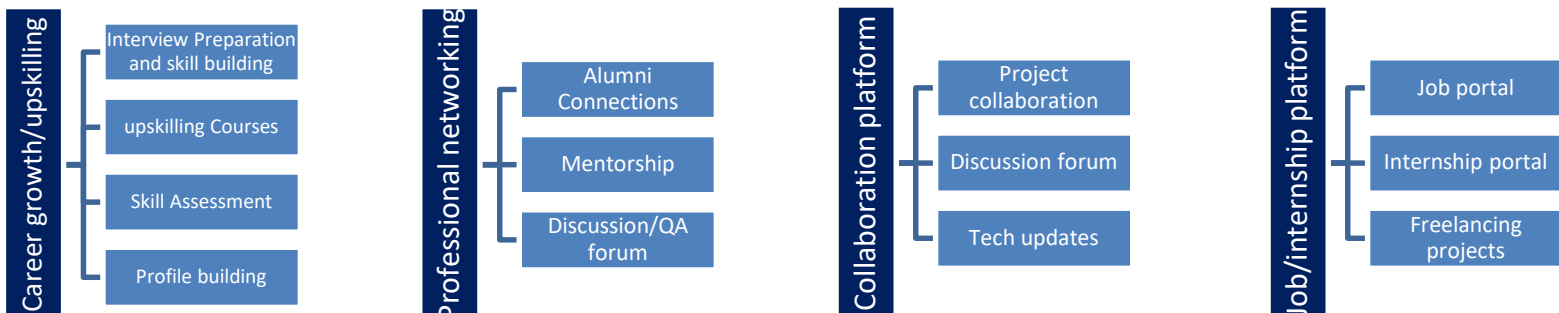
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

<https://www.upskillcampus.com/>





## 2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

## 2.4 Objectives of this Internship program

The objective for this internship program was to

- get practical experience of working in the industry.
- to solve real world problems.
- to have improved job prospects.
- to have Improved understanding of our field and its applications.
- to have Personal growth like better communication and problem solving.

## 2.5 Reference

- [1] <https://docs.python.org/3/>
- [2] <https://flask.palletsprojects.com/>
- [3] <https://www.sqlite.org/docs.html>
- [4] [https://www.w3schools.com/python/python\\_flask.asp](https://www.w3schools.com/python/python_flask.asp)

## 2.6 Glossary

| Terms | Acronym                           |
|-------|-----------------------------------|
| URL   | Uniform Resource Locator          |
| HTTP  | Hypertext Transfer Protocol       |
| SQL   | Structured Query Language         |
| API   | Application Programming Interface |
| Flask | Lightweight Python Web Framework  |

### 3 Problem Statement

With the increasing use of the internet, users frequently need to share web links through emails, social media platforms, and messaging applications. However, many URLs are long, complex, and difficult to remember, making them inconvenient to share and manage. Long URLs can also reduce readability and may exceed character limits on certain platforms.

The problem addressed in this project is the development of a **URL Shortener** that converts lengthy URLs into short, unique, and easy-to-share links. The system should validate the input URL, generate a unique short code, store the mapping between the original and shortened URLs in a database, and redirect users to the correct destination when the shortened URL is accessed. The proposed solution aims to provide a simple, efficient, and user-friendly application that improves link sharing while demonstrating the practical use of Python and Flask in web application development.

#### Existing and Proposed solution

The existing approach to URL sharing involves using the original, lengthy URLs directly or relying on third-party URL shortening services such as Bitly and TinyURL. While these services are widely used, they often require user registration for advanced features, impose limitations on free accounts, and may include advertisements or tracking mechanisms. Additionally, users have limited control over data storage, customization, and application behavior. This makes such services less suitable for educational purposes or organizations that require a simple, self-hosted URL shortening solution.

#### Limitations of the Existing System

- Long URLs are difficult to read, remember, and share.
- Many services require account registration for advanced features.
- Limited customization of shortened links.
- Dependence on third-party platforms.
- Privacy concerns due to external data storage and tracking.
- Free plans often have usage restrictions.

### **3.1 Code submission (Github link) :**

<https://github.com/Aditya-Commits-dev/upskillcampus/blob/main/URLShortner.py>

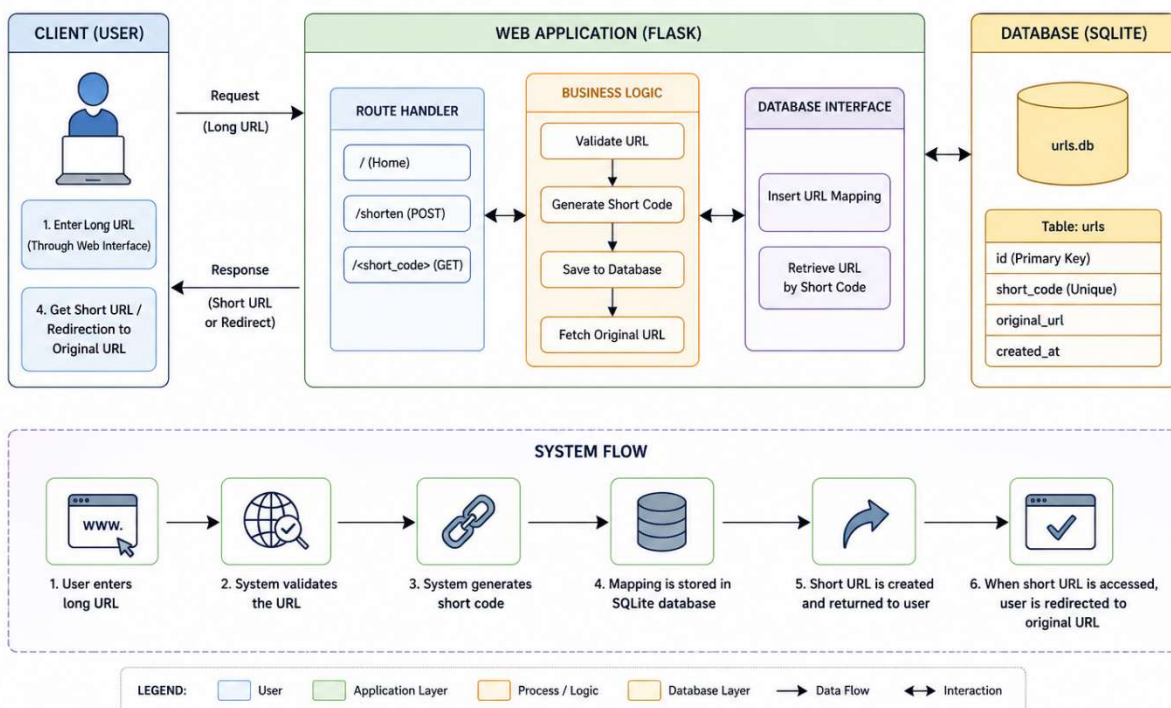
### **3.2 Report submission (Github link) :**

[https://github.com/Aditya-Commits-dev/upskillcampus/blob/main/URLShortener\\_Aditya\\_USC\\_UCT.pdf](https://github.com/Aditya-Commits-dev/upskillcampus/blob/main/URLShortener_Aditya_USC_UCT.pdf)

## 4 Proposed Design/ Model

The proposed **URL Shortener** is a lightweight web application developed using **Python, Flask, and SQLite**. It allows users to enter a long URL, validates the input, generates a unique short code, and stores the URL mapping in a local SQLite database. When a shortened URL is accessed, the system automatically redirects the user to the original webpage. The application is simple, fast, user-friendly, and easy to deploy, making it suitable for learning, small-scale use, and future enhancements.

### PROPOSED DESIGN MODEL DIAGRAM (URL SHORTENER)



## 5 Performance Test

### 5.1 Test Plan/ Test Cases

The purpose of performance testing was to evaluate the efficiency, responsiveness, and reliability of the URL Shortener application under normal operating conditions. The application was tested to ensure that it generates shortened URLs quickly, performs accurate redirection, validates user input, and maintains stable performance while processing multiple requests.

The following aspects were evaluated:

- URL generation speed
- URL redirection time
- Database response time
- URL validation accuracy
- Overall application responsiveness

| Test Case ID | Test Scenario                        | Expected Result                  | Actual Result             | Status |
|--------------|--------------------------------------|----------------------------------|---------------------------|--------|
| PT-01        | Generate short URL from valid URL    | Short URL generated successfully | Successfully generated    | Pass   |
| PT-02        | Access shortened URL                 | Redirect to original webpage     | Successfully redirected   | Pass   |
| PT-03        | Enter invalid URL                    | Validation error displayed       | Error message shown       | Pass   |
| PT-04        | Generate multiple URLs consecutively | Unique short URLs generated      | Generated successfully    | Pass   |
| PT-05        | Access non-existing short URL        | Display "URL Not Found" message  | Correct message displayed | Pass   |

## 5.2 Test Procedure

The application was executed on a local machine using the Flask development server. Different types of URLs, including valid, invalid, and duplicate URLs, were entered into the system. The generated short URLs were tested to verify successful database storage and accurate redirection. Multiple requests were also performed to evaluate the application's response time and stability.

The testing process included:

1. Launch the Flask application.
2. Enter a valid long URL.
3. Generate the shortened URL.
4. Open the shortened URL and verify redirection.
5. Test invalid URL inputs.
6. Generate multiple URLs to ensure uniqueness.
7. Verify application stability during repeated execution.

## 5.3 Performance Outcome

The performance testing showed that the application operates efficiently under normal usage conditions.

| Performance Metric          | Observed Result                      |
|-----------------------------|--------------------------------------|
| Average URL Generation Time | Less than 1 second                   |
| URL Redirection Time        | Instant (approximately 0.5–1 second) |
| URL Validation              | 100% Accurate                        |
| Database Storage            | Successful                           |
| System Stability            | Stable                               |
| Overall Performance         | Excellent                            |



## 6 Future work scope

The **URL Shortener** project successfully provides a simple and efficient solution for converting long URLs into short, easy-to-share links. Although the current system fulfills its primary objectives, several enhancements can be implemented in the future to improve its functionality, scalability, and security.

### 1. User Authentication

A user registration and login system can be integrated to allow users to manage their shortened URLs securely. Each user could have a personalized dashboard to view, edit, or delete their links.

### 2. Custom Short URLs

Instead of automatically generated short codes, users can be given the option to create custom aliases (e.g., mywebsite.com/aditya), making the links more meaningful and easier to remember.

### 3. QR Code Generation

The application can automatically generate a QR code for every shortened URL, allowing users to access links quickly by scanning them with mobile devices.

### 4. Click Analytics

Analytics features can be added to track the number of clicks, visitor locations, browser information, operating systems, and access times, helping users monitor link performance.

### 5. Link Expiration

Users can be provided with the option to set an expiration date or a maximum number of clicks for a shortened URL, after which the link becomes inactive automatically.

### 6. Cloud Deployment

The application can be deployed on cloud platforms such as **AWS, Microsoft Azure, Google Cloud Platform**, or **Render** to make it accessible from anywhere over the internet.

### 7. REST API Integration

A RESTful API can be developed to allow other applications, websites, or mobile apps to use the URL shortening service programmatically.

### 8. Enhanced Security

Security can be improved by implementing HTTPS support, spam detection, malicious URL filtering, rate limiting, and CAPTCHA verification to protect the application from misuse.

### **9. Admin Dashboard**

An administrative panel can be developed to monitor all generated URLs, manage users, view system statistics, and remove inappropriate or inactive links.

### **10. Scalable Database Support**

For handling large volumes of data, SQLite can be replaced with more scalable database systems such as **MySQL**, **PostgreSQL**, or **MongoDB**, improving performance for enterprise-level applications.